

Name: Manvith Reddy Kandi

Reg No: 24BCE10177

PROJECT : Fleet Dispatch Tracker

1. Technical Environment

- Language: PHP (Procedural/Lightweight OOP)
- Database: MongoDB (NoSQL)
- Testing Tool: Postman (for API verification)
- Package Manager: Composer (for MongoDB driver)

2. Objectives/Features:

1. DB Connections & Console-Based: Connects directly to MongoDB; core execution is driven by terminal commands (php cli.php).
2. No Frontend: Zero HTML, CSS, or JavaScript. Strictly a backend architecture.
3. API Checking: Includes an api.php router to process HTTP requests, outputting JSON for Postman verification.
4. Operations Point (Create): Implements Create/Insert functions for the 4 core entities: Drivers, Vehicles, Shipments, and Warehouses.
5. Full CRUD: Supports all required operations: Drop (Delete), Update, Insert, and Read across the database.
6. Insert Functionality: Direct database insertion methods are mapped to both CLI inputs and API POST requests.
7. Database Mappings (Relationships): Successfully implements all four relationship types using NoSQL document structures rather than SQL joins.

3. NoSQL Database Schema & Mappings

Instead of traditional relational tables, the project uses MongoDB's document architecture to prove an understanding of NoSQL database design:

- One-to-One (1:1): *Embedded Data*. A Vehicle document embeds its specific Registration_Details (capacity, fuel type) directly inside it.
- One-to-Many (1:N): *Parent Referencing*. A Warehouse document holds an array of vehicle_ids for all trucks currently parked there.
- Many-to-One (N:1): *Child Referencing*. Multiple individual Shipment documents reference a single warehouse_id where they are stored.
- Many-to-Many (M:N): *Junction Logging*. A Dispatch_Logs collection captures the exact driver_id, vehicle_id, and timestamp, mapping the reality that drivers operate multiple trucks and trucks are driven by multiple drivers over time.

4. Execution (How it Runs)

The system uses a shared database logic file (db.php) that is triggered in two different ways depending on the user's need:

- **Action:** Dispatching a Truck (Let)

Via Console (Internal Use):

- **Input:** `php cli.php dispatch "DRV_101" "TRK_55" "Warehouse_North"`
- **Output:** Terminal prints a success message and updates the database.

Via Postman (External API Use):

- **Input:** POST request to `http://localhost/api.php?action=dispatch` with JSON payload.
 - **Output:** Postman receives a formatted `{"status": "success"}` JSON response, and the database is updated.